15.1. Hashing and Hash Functions

Previous Section:

15. Hash Tables

Contents:

In the previous section, we learned that **Hash Tables** can locate data extremely quickly by storing information according to an index. But this raises an important question:

> **How does the computer know which index to use?**

The answer lies in two fundamental concepts: **Hashing** and **Hash Functions**.

Hashing is the process of transforming a piece of data into a fixed-size representation, while a Hash Function is the mathematical algorithm that performs this transformation. Together, they allow computers to determine where data should be stored and retrieved without searching through every element.

1. What is Hashing?

Hashing is the process of converting data of **any size or length** into a value of **fixed length** using a mathematical algorithm known as a **Hash Function**.

No matter whether the input is a single word, an image, a document, or a large video file, the output is always a hash value of a predetermined size.

For example, consider the **MD5** hashing algorithm: The word **"Apple";** A photograph; A two-hour video

Although these inputs have completely different sizes, MD5 always produces a **32-character hexadecimal string**.

For example,

```
Input
+---------+      +------------------+      +----------------------------------+
|  Apple  | ---> |   MD5 Hashing    | ---> | 1F3870BE274F6C49B3E31A0C6728957F |
+---------+      +------------------+      +----------------------------------+
```

Likewise,

```
+--------+      +-----------+      +--------------------+
| Photo  | ---> |    MD5    | ---> | 32-character Hash  |
+--------+      +-----------+      +--------------------+

+--------+      +-----------+      +--------------------+
| Video  | ---> |    MD5    | ---> | 32-character Hash  |
+--------+      +-----------+      +--------------------+

+--------+      +-----------+      +--------------------+
| Apple  | ---> |    MD5    | ---> | 32-character Hash  |
+--------+      +-----------+      +--------------------+
```

One important property of hashing is that it is **one-way**.

```
Original Data ---> Hash Function ---> Hash Value
      ✓                                   ✓
 Easy to Compute                  Fixed-Length Output

Hash Value --------X--------> Original Data
             (Cannot Recover)
```

Given the original data, computing the hash value is straightforward. However, given only the hash value, **it is practically impossible to reconstruct the original input.**

This characteristic makes hashing extremely useful for fast data retrieval; password storage; data integrity verification; digital signatures; cryptography

1.1. Hashing in Data Structures

When Hashing is used together with a Hash Table, the purpose is no longer security, but **fast searching**.

Instead of searching every element sequentially, we compute the position where the data should be stored.

```
                +-----------------+
Key (45) --->   | Hash Function  | ---> Index (6) ---> Bucket 6
                +-----------------+
```

Because the correct position can be calculated immediately, searching usually takes **Time Complexity** $= O(1)$

Hashing is therefore ideal for applications that perform frequent searching.

However, if the application frequently inserts or deletes data while maintaining a sorted order, balanced binary search trees are often a better choice.

2. What is a Hash Function?

A **Hash Function** is the mathematical algorithm that performs the hashing process.

Its job is to convert a key into a fixed-size value, which is then used as an index inside a Hash Table.

```
                        +-----------------+
    Key  --->   | Hash Function  | ---> Hash Value ---> Has
h Table Index
                        +-----------------+
```

In other words: **Hashing** is the process, while the **Hash Function** is the algorithm that performs the process.

2.1. Library Analogy

Imagine a large library containing thousands of books. Each book has two important pieces of information: A **Unique Book ID;** A **Call Number** indicating its shelf location.

```
Book
+------------------------+
| Book ID : 418273       | -- Locate Shelf -- QA76.73
| Call No.: QA76.73      |
+------------------------+
```

The librarian does not search every shelf one by one. Instead, the call number immediately tells where the book belongs.

A Hash Table works in a very similar way. The Hash Function determines where every key should be stored.

```
                       +----------------+
Student ID = 1025  ---> | Hash Function  | ---> Index = 8 ---> Store at Bucket 8
                       +----------------+
```

2.2. Collisions

Ideally, every key should map to a different index. However, since many different keys may exist while the number of buckets is limited, two or more keys may produce the same index.

This situation is known as a **collision**.

```
Key 18 ----\
            \  +----------------+
Key 39 ------> | Hash Function  | -----> Index 4
            /  +----------------+
Key 53 ----/
```

All three keys point to the same bucket *(Index 4)*.

Since collisions cannot be completely avoided, every Hash Table must include a method for handling them, which will be discussed in the following sections.

2.3. Characteristics of a Good Hash Function

A good Hash Function should satisfy several important properties:

- **Fast to compute**, so that searching remains efficient.
- **Produces very few collisions**, reducing conflicts between keys.
- **Distributes keys uniformly** across the Hash Table, avoiding clusters where many keys are stored in the same bucket.
- **Produces deterministic results**, meaning the same input always generates the same output.

A well-designed Hash Function allows Hash Tables to achieve their average **O(1)** search performance while making efficient use of available storage.

3. Implementation of Hashing

Once we understand what hashing and hash functions are, the next step is learning **how they are implemented in practice**.

Overall Hashing Process:

```
+------+      +------------+      +-----------------+      +-------------+      +------
----------------+
| Key  | ---> | Hash Code  | ---> | Hash Function  | ---> | Table Index | ---> | Store
Key-Value Pair |
+------+      +------------+      +-----------------+      +-------------+      +------
----------------+
```

The general idea is straightforward:

1. Convert a key into a numerical value (called the **hash code**).
2. Use a hash function to compute the corresponding table index.
3. Store the key-value pair at that index.
4. If multiple keys map to the same index, resolve the resulting **collision** using an appropriate collision resolution technique.

3.1. Hash Function Selection

There is no single hash function suitable for every application. Depending on the type of data being stored, different hashing techniques can be used.

Some of the most common methods include:

- Modular Arithmetic: The simplest and most widely used method is **Modular Arithmetic**, where the index is computed using the modulo operator.

$$\text{Index} = k \bmod C$$

Where: k is the hash code (or key), C is the size of the hash table, usually chosen as a **prime number** to improve key distribution.

Example:

```
+-----------+      +----------------+      +-------------+      +-----------+
| Key = 321 | ---> |   321 mod 10   | ---> |  Index = 1  | ---> | Bucket 1  |
+-----------+      +----------------+      +-------------+      +-----------+
```

Because of its simplicity and efficiency, this method is commonly used in introductory implementations of Hash Tables.

- **Truncation:** The **Truncation** method forms the index by selecting a portion of the key.

For example, suppose the key is `1223456689`, we may simply use `1223` as the index (or as the value used to compute the final index).

Although easy to implement, truncation may not distribute keys uniformly if many keys share similar prefixes or suffixes.

```
+--------------+      +----------------+      +---------------+
| 1223456689   | ---> | Take "1223"    | ---> | Compute Index |
+--------------+      +----------------+      +---------------+
```

- **Folding:** The **Folding** method divides a long key into several smaller parts and combines them to produce the hash code.

For example, given the key `1223456689`, we may divide it into `1223`, `456`, `689`

The parts are then added together before applying the modulo operation: $(1223 + 456 + 689) \bmod C$

```
+--------------+      +----------------------+      +------------------+      +-------
------+
| 1223456689   | ---> | 1223 + 456 + 689     | ---> | (2368 mod C)     | ---> | Table
Index |
+--------------+      +----------------------+      +------------------+      +-------
------+
```

Folding often produces a better distribution than truncation because it considers multiple portions of the key rather than only one.

- **Example: Integer Keys**

To illustrate the hashing process, consider the following collection of key-value pairs: `{100 : "Paris", 123 : "Manchester", 321 : "Barcelona", 555 : "Milan", 777 : "Dortmund"}`

Since all keys are already integers, they can be used directly as the **hash codes**.

Suppose the hash table contains **10 buckets**. The hash function is $\text{Index} = \text{Key} \bmod 10$

The resulting indices are:

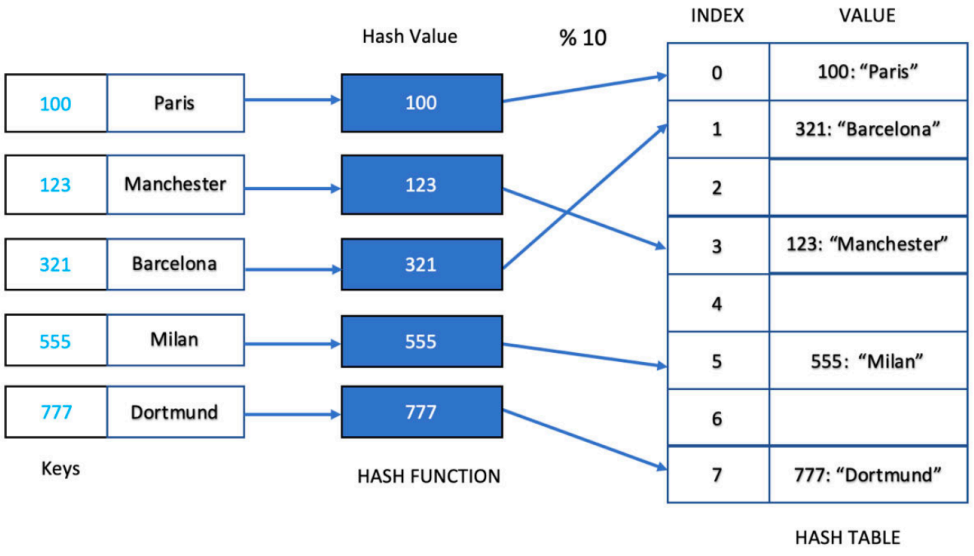
```
+-----------+      +----------------+      +-------------+
| Key = 100 | ---> | 100 mod 10     | ---> | Index = 0   |
+-----------+      +----------------+      +-------------+

+-----------+      +----------------+      +-------------+
| Key = 123 | ---> | 123 mod 10     | ---> | Index = 3   |
+-----------+      +----------------+      +-------------+

+-----------+      +----------------+      +-------------+
| Key = 321 | ---> | 321 mod 10     | ---> | Index = 1   |
+-----------+      +----------------+      +-------------+

+-----------+      +----------------+      +-------------+
| Key = 555 | ---> | 555 mod 10     | ---> | Index = 5   |
+-----------+      +----------------+      +-------------+

+-----------+      +----------------+      +-------------+
| Key = 777 | ---> | 777 mod 10     | ---> | Index = 7   |
+-----------+      +----------------+      +-------------+
```



Each key-value pair is then stored at its corresponding index.

- **Example: String Keys**

In practice, keys are often **strings** rather than integers. For example, `{"100" : "Paris", "123" : "Manchester", "321" : "Barcelona", "555" : "Milan", "777" : "Dortmund"}`

Since strings cannot be directly used with modular arithmetic, they must first be converted into an integer hash code.

```
+-------------+      +-----------------+      +----------------+      +-------------+
| Key = "100" | ---> | ASCII Hash Code | ---> | 48625 mod 10   | ---> | Index = 5   |
+-------------+      +-----------------+      +----------------+      +-------------+
```

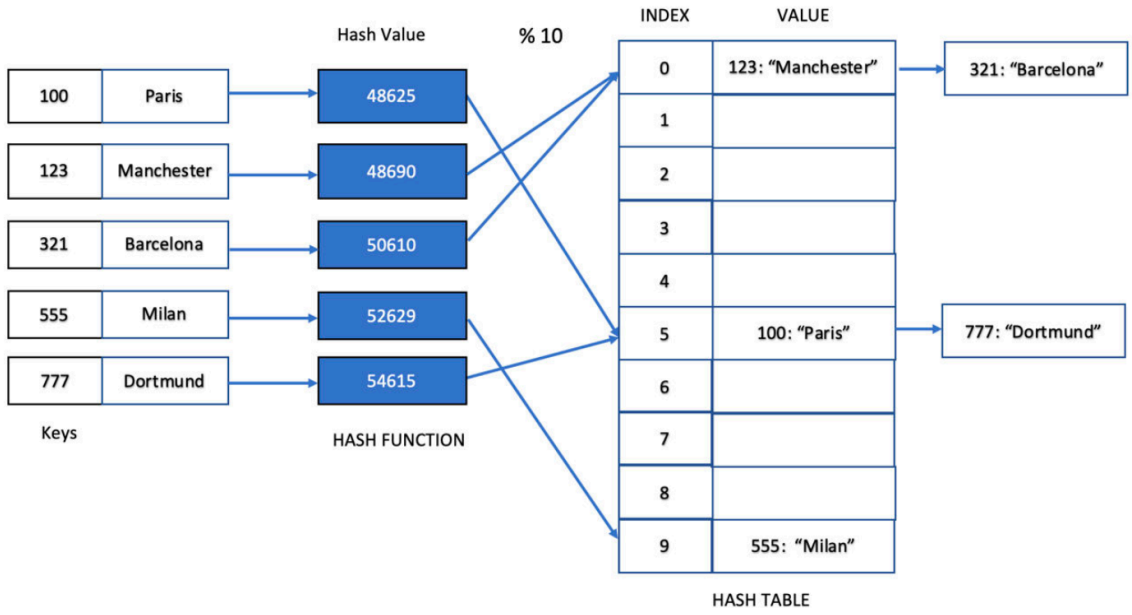
A commonly used polynomial hash formula is

$$\text{Hash Code} = \text{ord}(s_0) \times 31^{n-1} + \text{ord}(s_1) \times 31^{n-2} + \cdots + \text{ord}(s_{n-1}) \times 31^0$$

where: n is the number of characters, s_i is the i^{th} character, $ord()$ returns the ASCII value of a character (as in Python).

For example, "100" is converted into $49 \times 31^2 + 48 \times 31^1 + 48 \times 31^0 = 48625$. Similarly,

Key	Hash Code
"100"	48625
"123"	48690
"321"	50610
"555"	52629
"777"	54615

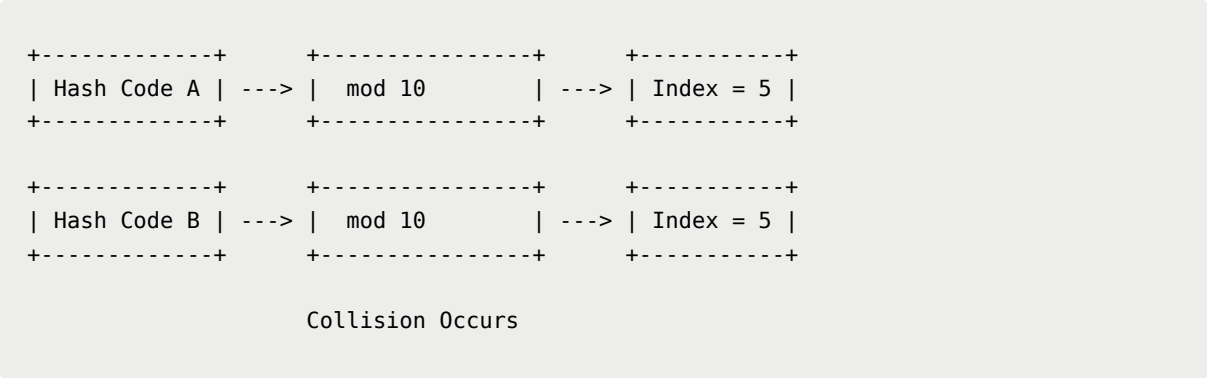


Separate Chaining is performed for this hash table

These integer hash codes are then passed through the chosen hash function to determine their positions in the Hash Table.

- **Collisions**

During hashing, it is possible for multiple keys to produce the same table index.



This situation is called a **collision**.

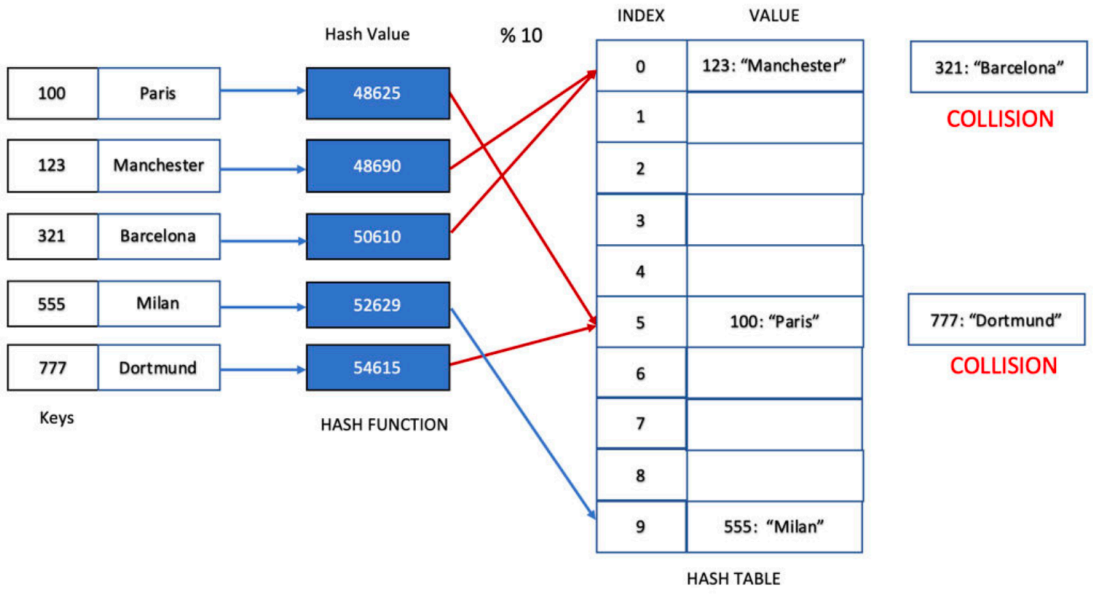
Collisions become increasingly common as the number of stored elements grows or when the table size is too small.

Whenever a collision occurs, the Hash Table cannot simply overwrite the existing value. Instead, it must use a **collision resolution technique** to ensure that every key-value pair remains accessible.

Several collision resolution methods have been developed, each with its own advantages and trade-offs.

3.2. Collision Resolution

A **collision** occurs when two or more keys are mapped to the same index in a Hash Table.



Since every key-value pair must still be stored correctly, the Hash Table requires a strategy for handling these conflicts.

The most common collision resolution techniques are described below.

- **Increase the Hash Table Size:** The simplest solution is to increase the size of the Hash Table and perform the hashing process again.

For example, if many collisions occur using $k \bmod 5$, we may increase the table size and use

$k \bmod 10, \quad k \bmod 15, \quad k \bmod 20$ or another larger size.

```
+---------+      +----------------------+      +----------+
| mod 5   | ---> | Too Many Collisions  | ---> | mod 10   |
+---------+      +----------------------+      +----------+
```

Although simple, this approach is generally inefficient because every existing element must be hashed again whenever the table size changes.

- **Open Addressing**

Instead of storing multiple values at the same index, **Open Addressing** searches for another available position within the table.

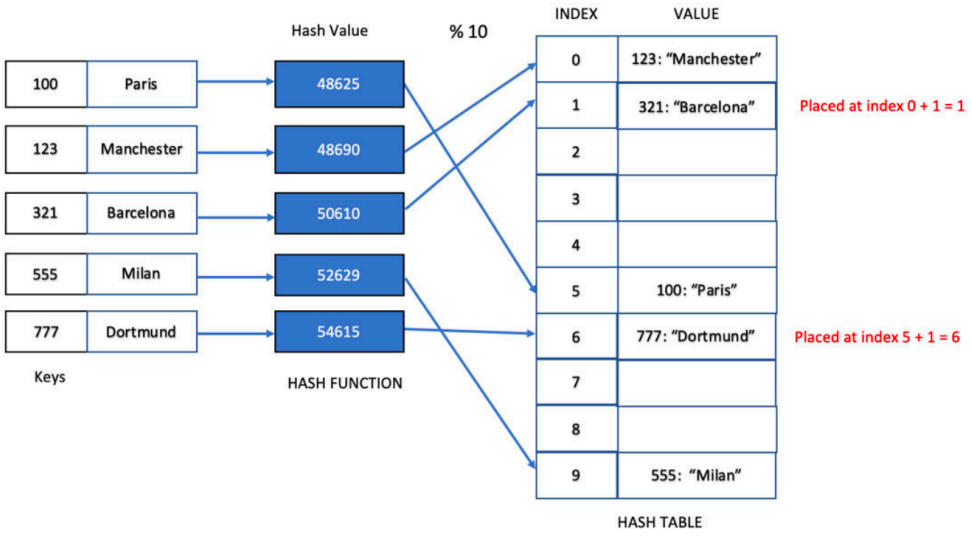
```
+-----------+      +-----------------+      +-----------------+
| Index = 5 | ---> | Bucket Occupied | ---> | Search Next Slot|
```

```
+-----------+      +-----------------+      +-----------------+
```

Several probing strategies are commonly used.

Linear Probing: Searches the table sequentially until an empty position is found. The next candidate position is computed as $\text{New Index} = \text{Current Index} + f(i)$ where $f(i) = i$.

```
+-----------+      +-----------+      +-----------+      +------------+
| Bucket 5  | ---> | Bucket 6  | ---> | Bucket 7  | ---> | Empty Slot |
| Occupied  |      | Occupied  |      | Occupied  |      |  Insert    |
+-----------+      +-----------+      +-----------+      +------------+
```

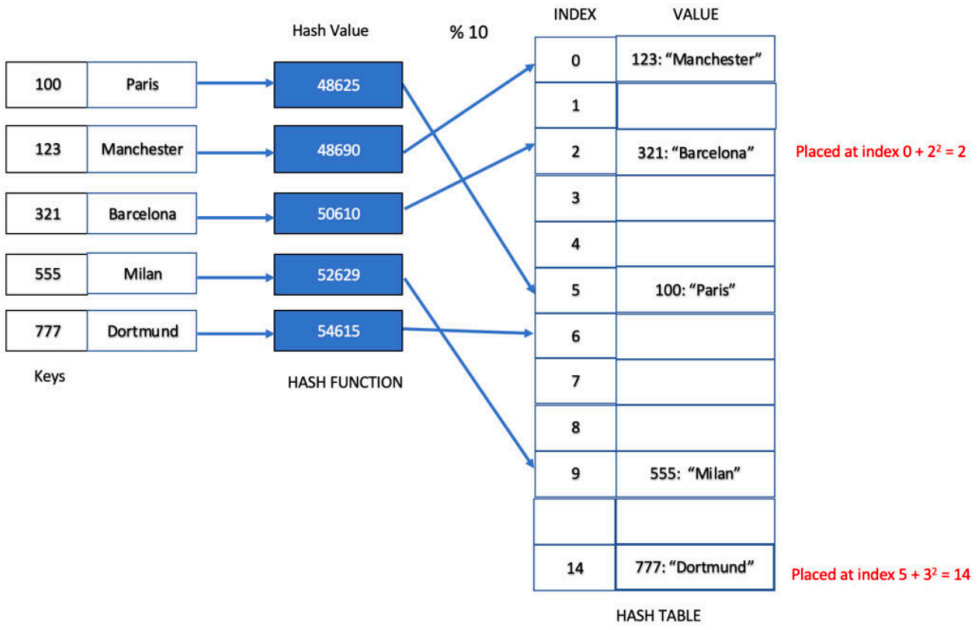


Linear Probing : Index = Index + 1, Index + 2, etc.

This method is straightforward and easy to implement.

However, it suffers from **Primary Clustering**, where groups of consecutive occupied positions begin forming. As these clusters grow larger, insertion and searching become increasingly inefficient because more positions must be examined.

Quadratic Probing reduces clustering by increasing the probe distance quadratically. The probing function becomes $f(i) = i^2$ and the next position is calculated as $\text{New Index} = \text{Current Index} + i^2$.



Quadric Probing : Index = Index + 1^2, Index + 2^2, etc.

Because the probe sequence spreads out more rapidly than Linear Probing, clustering is significantly reduced.

- **Re-Hashing**

Another approach is to use a **second Hash Function** whenever a collision occurs.

Instead of searching nearby positions, a new index is computed using another independent function. For example, $h_2(k) = 7 - (k \bmod 7)$.

Suppose a collision occurs after applying the primary hash function. The second hash function computes an alternative index, allowing the new value to be stored elsewhere in the table.

Using multiple hash functions generally produces a better distribution than simple probing and reduces clustering.

- **Separate Chaining**

Separate Chaining stores all colliding elements at the same table index. Instead of placing only one value in each bucket, every bucket stores a **linked list** (or another dynamic collection).

Whenever a collision occurs, the new key-value pair is simply inserted into the linked list associated with that bucket.

Separate Chaining is one of the most widely used collision resolution techniques because it is simple to implement and can accommodate an unlimited number of collisions at a single index. The trade-off is that searching within a heavily populated bucket becomes a linear-time operation proportional to the length of the chain.

Among all collision resolution methods, Separate Chaining is often preferred when the expected number of collisions is relatively high, while Open Addressing techniques are generally favored when memory usage is a primary concern.

3.3. Implementation

The following code implements the Hash Table class with some key methods.

- Node for Separate Chaining

```
class Node:
    def __init__(self, key):
        self.key = key
        self.next = None
```

- The HashTable Class

```
class HashTable:
    def __init__(self, size):
        self.size = size

    # Utility Methods
    def get_length(self, data):
        return len(data)

    def get_max(self, dictionary):
        return max(dictionary.values())

    def hash_code(self, string):
```

```
        result = 0
        n = len(string)

        for i in range(n):
            result += ord(string[i]) * (31 ** (n - i - 1))
        return result

    def hash_function(self, key):
        return key % self.size

    def hash_strings(self, strings):
        table = {}

        for word in strings:
            code = self.hash_code(word)
            table[code] = self.hash_function(code)
        return table
```

- Separate Chaining

```
class SeparateChaining(HashTable):
    def __init__(self, size):
        super().__init__(size)
        self.table = [None] * size

    def insert(self, key):
        index = self.hash_function(key)

        if self.table[index] is None:
            self.table[index] = Node(key)
        else:
            current = self.table[index]
            while current.next is not None:
                current = current.next
            current.next = Node(key)

    def count_collisions(self):
        collisions = 0
        for slot in self.table:
            if slot is None:
                continue

            count = 0
            current = slot

            while current:
                count += 1
                current = current.next

            if count > 1:
                collisions += count - 1

        return collisions

    def display(self):
        for i in range(self.size):
            print(f"Hash Table[{i}]: ", end="")
            current = self.table[i]
```

```
            if current is None:
                print("None")
                continue

            while current:
                print(current.key, end=" -> ")
                current = current.next

            print("None")


keys = [25, 43, 13, 26, 44, 35, 20]
# Separate Chaining
chain = SeparateChaining(10)

for key in keys:
    chain.insert(key)
chain.display()

print("+ Collisions:", chain.count_collisions())
```

```
Hash Table[0]: 20 -> None
Hash Table[1]: None
Hash Table[2]: None
Hash Table[3]: 43 -> 13 -> None
Hash Table[4]: 44 -> None
Hash Table[5]: 25 -> 35 -> None
Hash Table[6]: 26 -> None
Hash Table[7]: None
Hash Table[8]: None
Hash Table[9]: None
Collisions: 2
```

- Open-Addressing

```
class OpenAddressing(HashTable):
    def __init__(self, size):
        super().__init__(size)
        self.table = [0] * size

    def linear_probing(self, key):
        collisions = 0
        index = self.hash_function(key)

        while self.table[index] != 0:
            index = (index + 1) % self.size
            collisions += 1

        self.table[index] = key
        return collisions

    def quadratic_probing(self, key):
        collisions = 0

        original = self.hash_function(key)
        index = original
        i = 1

        while self.table[index] != 0:
            index = (original + i * i) % self.size
            collisions += 1
            i += 1

        self.table[index] = key
        return collisions

    def display(self):
        for i, value in enumerate(self.table):
            print(f"Hash Table[{i}]: {value}")


keys = [25, 43, 13, 26, 44, 35, 20]
```

```
# Linear Probing
print("-" * 50)
print("Linear Probing")
print("-" * 50)

# Note that you must choose a value equal
# or greater than the number of keys
linear = OpenAddressing(10)

for key in keys:
    linear.linear_probing(key)

linear.display()


# Quadratic Probing
print()
print("-" * 50)
print("Quadratic Probing")
print("-" * 50)

# Note that you must choose a value equal
# or greater than the number of keys
quadratic = OpenAddressing(10)

for key in keys:
    quadratic.quadratic_probing(key)

quadratic.display()
```

```
--------------------------------------------------
Linear Probing
--------------------------------------------------
Hash Table[0]: 20
Hash Table[1]: 0
Hash Table[2]: 0
Hash Table[3]: 43
Hash Table[4]: 13
Hash Table[5]: 25
Hash Table[6]: 26
```

```
Hash Table[7]: 44
Hash Table[8]: 35
Hash Table[9]: 0

------------------------------------------------
Quadratic Probing
------------------------------------------------
Hash Table[0]: 20
Hash Table[1]: 0
Hash Table[2]: 0
Hash Table[3]: 43
Hash Table[4]: 13
Hash Table[5]: 25
Hash Table[6]: 26
Hash Table[7]: 0
Hash Table[8]: 44
Hash Table[9]: 35
```

Note: One limitation of **Quadratic Probing** is that it typically requires the **Hash Table size to be carefully chosen** (usually a prime number) and may require **dynamic resizing (rehashing)** as the table becomes full.
Without resizing, Quadratic Probing may fail to find an empty bucket even when free slots still exist, causing insertion to fail despite available space.

Summary

Hashing is the process of converting data of any size into a fixed-length hash value using a **Hash Function**, allowing data to be stored and retrieved efficiently in a **Hash Table**.

A good Hash Function distributes keys uniformly, minimizes collisions, and enables fast access to data, typically in $O(1)$ average time.

Common hash function techniques include **Modular Arithmetic**, **Truncation**, and **Folding**, while string keys are first converted into integer hash codes before hashing.

Since different keys may produce the same index, collisions are unavoidable and must be resolved using techniques such as **Open Addressing** (Linear or Quadratic Probing), **Re-Hashing**, or **Separate Chaining**.

Together, hashing and collision resolution form the foundation of efficient Hash Table implementation.

References

https://drive.google.com/file/d/1le3JCsQzFeq4MydZEYR2XLKUlSUaUypv/view?usp=drive_link

https://www.geeksforgeeks.org/dsa/introduction-to-hashing-2/

https://www.geeksforgeeks.org/dsa/hash-functions-and-list-types-of-hash-functions/

Next Section:

16. String Matching